



EstateEase Platform

Real Estate Ecosystem Architecture

FULL SYSTEM SPECIFICATION & FILE REFERENCE

EstateEase Core Website Architecture Manual

A comprehensive technical breakdown of the Buyer Panel, Seller Workspace, Admin Command Center, Public Portal, REST API Controllers, Database Schemas, and Real-Time WebSockets.

1. Platform Architecture & Multi-Role Ecosystem

The **EstateEase Real Estate Platform** is built as a modern, decoupled full-stack web application. It seamlessly connects property seekers (Buyers), property owners/agents (Sellers), and system administrators (Admins) within a unified real-time ecosystem.

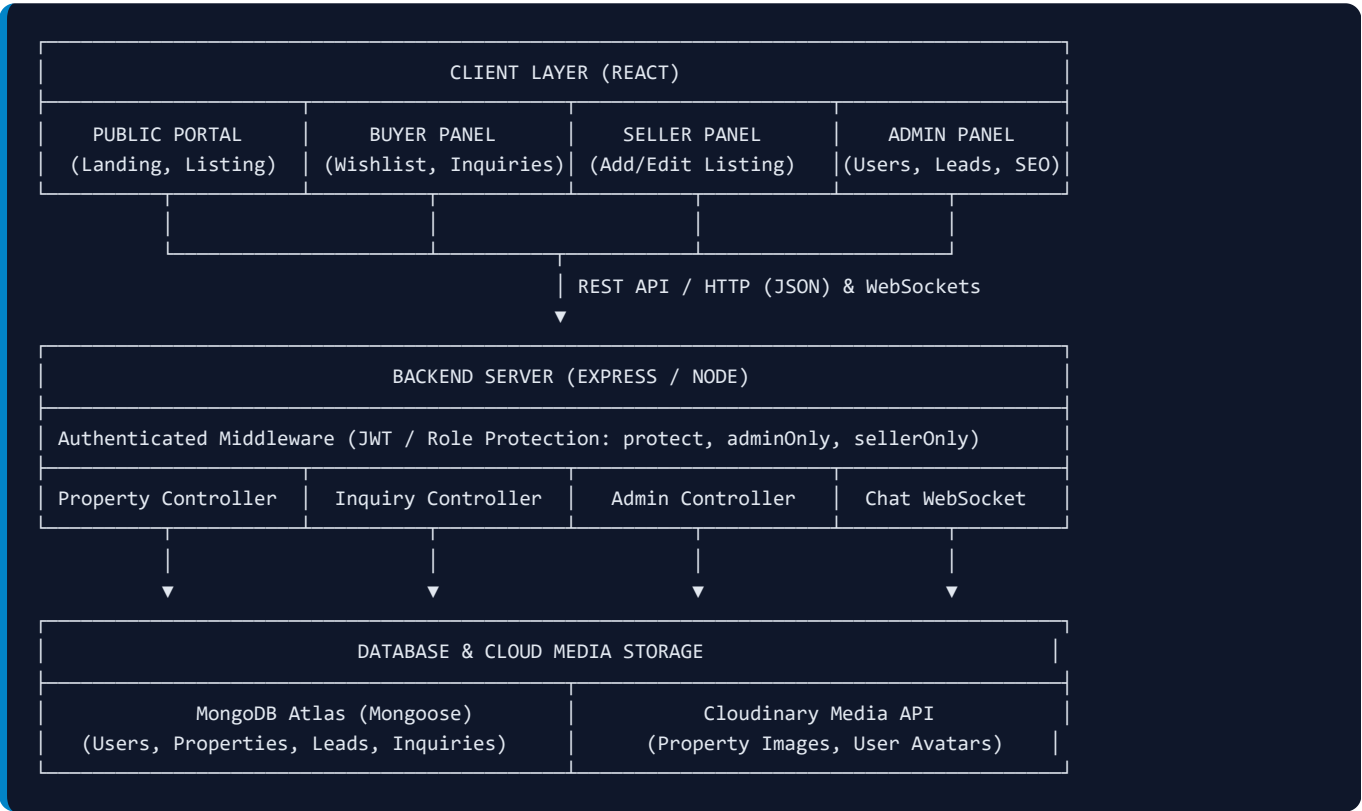
Frontend Stack (Client)

React (Vite), React Router v6, Tailwind CSS, Lucide / React Icons, Axios, Socket.io-client, Context API (AuthContext).

Backend Stack (Server)

Node.js, Express.js (v5), MongoDB + Mongoose ODM, Socket.io Server, Cloudinary API, JWT Auth, BcryptJS, Multer.

High-Level Architectural Diagram



2. Buyer Panel Architecture & Component Reference

The Buyer Panel allows registered users to search properties, filter by multi-variable specifications, save favorite properties to a personal wishlist, send direct property inquiries to sellers, and communicate in real-time.

FILE PATH	TYPE	SPECIFIC PURPOSE & IMPLEMENTATION DETAILS
<code>frontend/src/pages/buyer/Wishlist.jsx</code>	Page Component	Renders the user's saved properties grid. Communicates with <code>/api/wishlist</code> to fetch, add, or remove property bookmarks dynamically with real-time UI updates.
<code>frontend/src/pages/buyer/MyInquiries.jsx</code>	Page Component	Displays a history of inquiries sent by the buyer to landlords/agents. Shows inquiry status (<code>pending</code> , <code>contacted</code> , <code>closed</code>), message history, and property reference cards.
<code>frontend/src/pages/shared/properties.jsx</code>	Page Component	Advanced property catalogue featuring multi-variable filters (city, locality, price range, property type, BHK, sorting by price/date) and responsive pagination.
<code>frontend/src/pages/shared/PropertyDetails.jsx</code>	Page Component	Detailed view for individual listings featuring image carousel gallery, amenity badges, location details, price analysis, seller details card, and direct contact inquiry form.
<code>frontend/src/pages/shared/Profile.jsx</code>	Page Component	User profile management interface for updating name, email, phone number, password, and uploading profile avatars via Cloudinary.

3. Seller Panel Architecture & Component Reference

The Seller Panel provides real estate owners and agents with a dedicated dashboard to manage their listings, monitor buyer inquiries, track views, and publish new properties.

FILE PATH	TYPE	SPECIFIC PURPOSE & IMPLEMENTATION DETAILS
<code>frontend/src/pages/seller/SellerDashboard.jsx</code>	Page Component	Main seller control center displaying listing performance metrics (total listings, active status, pending approvals, inquiry counts) and quick navigation action cards.
<code>frontend/src/pages/seller/AddProperty.jsx</code>	Page Component	Multi-step property listing creation wizard. Handles property metadata (title, description, propertyType, price, city, locality, address, bedrooms, bathrooms, areaSqFt, amenities) and drag-and-drop image uploads to Cloudinary.

FILE PATH	TYPE	SPECIFIC PURPOSE & IMPLEMENTATION DETAILS
frontend/src/pages/seller/EditProperty.jsx	Page Component	Property editing interface allowing sellers to update listing specifications, modify pricing, toggle availability (<code>available</code> , <code>sold</code> , <code>rented</code>), and add/delete photo gallery images.
frontend/src/pages/seller/MyProperties.jsx	Page Component	Management table of all listings owned by the seller. Features quick status toggling, view count metrics, edit shortcuts, and property deletion with confirmation modal.
frontend/src/pages/seller/PendingApproval.jsx	Page Component	Onboarding status screen displayed to buyers who requested seller role elevation while their request is pending administrative review.
frontend/src/components/SellerLayout.jsx	Layout Component	Wrapper component rendering the dedicated seller navigation header, breadcrumbs, and responsive sidebar framework.
frontend/src/components/SellerSidebar.jsx	Navigation Component	Sidebar containing seller navigation links (Dashboard, Add Property, My Listings, Inquiries, Profile) with active state highlighting.

4. Admin Panel Architecture & Component Reference

The Admin Panel gives system administrators complete control over platform operations, user account roles, listing moderation, lead CRM, contact inquiries, and SEO metadata.

FILE PATH	TYPE	SPECIFIC PURPOSE & IMPLEMENTATION DETAILS
<code>frontend/src/pages/admin/AdminDashboard.jsx</code>	Page Component	Command center overview displaying platform-wide analytics: total registered users, active properties, pending seller applications, captured leads, and server health.
<code>frontend/src/pages/admin/AdminUsers.jsx</code>	Page Component	User management table for searching, filtering, editing user roles (<code>buyer</code> , <code>seller</code> , <code>admin</code>), activating/deactivating accounts, and deleting users.
<code>frontend/src/pages/admin/SellerRequests.jsx</code>	Page Component	Onboarding approval queue for evaluating seller join applications (<code>joinRequest.model.js</code>). Features one-click approval (elevates role to <code>seller</code>) or rejection with feedback.
<code>frontend/src/pages/admin/AdminProperties.jsx</code>	Page Component	Platform-wide listing moderation suite. Enables admins to approve pending seller listings, feature listings on the homepage hero, or remove inappropriate posts.
<code>frontend/src/pages/admin/AdminLeads.jsx</code>	Page Component	Lead management CRM interface. Filters captured leads by status (<code>new</code> , <code>contacted</code> , <code>qualified</code> , <code>closed</code>), intent, and city. Includes CSV export functionality.
<code>frontend/src/pages/admin/AdminInquiries.jsx</code>	Page Component	Centralized monitor for all buyer-seller direct inquiries across the platform to ensure service quality and seller response times.
<code>frontend/src/pages/admin/AdminContacts.jsx</code>	Page Component	Submissions queue for messages sent through the public "Contact Us" form (<code>contact.model.js</code>) with status marking (<code>unread</code> / <code>read</code>).
<code>frontend/src/pages/admin/AdminSEO.jsx</code>	Page Component	Dynamic SEO meta manager allowing admins to configure page titles, meta descriptions, canonical URLs, and OpenGraph social preview images for any route.
<code>frontend/src/pages/admin/AdminChat.jsx</code>	Page Component	Live support chat desk enabling admins to view active buyer-seller message threads and intervene if support assistance is requested.

FILE PATH	TYPE	SPECIFIC PURPOSE & IMPLEMENTATION DETAILS
<code>frontend/src/components/AdminLayout.jsx</code> & <code>AdminSidebar.jsx</code>	Layout Components	Administrative dashboard framework with responsive collapsible sidebar and top header metrics.

5. Public Portal, Shared Pages & Authentication Components

Public Landing & Shared Pages <code>LandingPage.jsx</code> – Hero search, city cards, stats counters, infinity loop logo carousel, testimonials. <code>About.jsx</code> – Mission, company history, leadership team, achievement metrics. <code>Contact.jsx</code> – Contact form, office location details, phone/email support links. <code>ChatMessages.jsx</code> – Real-time 1-on-1 messaging interface powered by Socket.io.	Authentication Pages (<code>auth/</code>) <code>Login.jsx</code> – Email/password login with JWT token issuance. <code>Register.jsx</code> – User registration with password hashing & verification. <code>VerifyEmail.jsx</code> – Email verification link processor. <code>ForgotPassword.jsx</code> & <code>ResetPassword.jsx</code> – Password reset token flow.
--	---

6. Backend Infrastructure, Database Models & API Controllers

6.1 Database Schemas & Models ([backend/models/](#))

MODEL FILE	COLLECTION	FIELDS & PURPOSE
<code>user.model.js</code>	<code>users</code>	<code>name, email, password, role ('buyer' 'seller' 'admin'), phone, avatar, isVerified, createdAt</code>
<code>property.models.js</code>	<code>properties</code>	<code>title, description, propertyType, price, location { city, locality, address, coordinates }, bedrooms, bathrooms, areaSqFt, images [], owner (Ref: User), status ('pending' 'available' 'sold' 'rented'), views</code>
<code>lead.model.js</code>	<code>leads</code>	<code>name, email, phone, intent, city, locality, propertyType, bedrooms, budget, timeline, status ('new' 'contacted' 'qualified' 'closed'), assignedTo</code>
<code>inquiry.model.js</code>	<code>inquiries</code>	<code>property (Ref: Property), buyer (Ref: User), seller (Ref: User), message, phone, status</code>
<code>joinRequest.model.js</code>	<code>joinrequests</code>	<code>user (Ref: User), agencyName, phone, reason, status ('pending' 'approved' 'rejected')</code>
<code>conversation.model.js</code> & <code>message.model.js</code>	<code>conversations</code> / <code>messages</code>	<code>participants [], lastMessage, unreadCount / conversationId, sender, text, attachments [], isRead</code>
<code>wishlist.model.js</code>	<code>wishlists</code>	<code>user (Ref: User), properties [Ref: Property]</code>
<code>contact.model.js</code> & <code>seoSetting.model.js</code>	<code>contacts</code> / <code>seosettings</code>	Public contact submissions & route-based dynamic SEO meta configurations.

6.2 Controllers & API Routes Mapping

CONTROLLER FILE	ROUTE FILE	HTTP ENDPOINTS & CORE RESPONSIBILITIES
<code>auth.controller.js</code>	<code>auth.routes.js</code>	<code>POST /api/auth/register</code> , <code>POST /api/auth/login</code> , <code>GET /api/auth/me</code> . Password hashing with bcrypt, JWT token generation.
<code>property.controller.js</code>	<code>property.routes.js</code>	<code>GET /api/properties</code> , <code>POST /api/properties</code> , <code>GET /api/properties/:id</code> , <code>PUT /api/properties/:id</code> , <code>DELETE /api/properties/:id</code> . Multi-parameter filtering, Cloudinary image upload middleware.
<code>admin.controller.js</code>	<code>admin.routes.js</code>	<code>GET /api/admin/stats</code> , <code>GET /api/admin/users</code> , <code>PATCH /api/admin/users/:id/role</code> , <code>GET /api/admin/seller-requests</code> , <code>PATCH /api/admin/seller-requests/:id</code> . Role-protected administration.

CONTROLLER FILE	ROUTE FILE	HTTP ENDPOINTS & CORE RESPONSIBILITIES
lead.controller.js	lead.routes.js	GET /api/leads , POST /api/leads , PATCH /api/leads/:id , DELETE /api/leads/:id . Lead capture CRUD & status pipeline management.
inquiry.controller.js	inquiry.routes.js	POST /api/inquiries , GET /api/inquiries/my-inquiries . Direct property inquiry creation & seller notification.
conversation.controller.js	conversation.routes.js	GET /api/conversations , POST /api/conversations/message . Real-time message storage and read state management.

7. End-to-End System Workflows & Interconnectivity

- 1

Property Listing & Moderation Workflow

Seller creates listing in `AddProperty.jsx` \$ ightarrow\$ Cloudinary handles photo uploads \$ ightarrow\$ Property created with status `'pending'` \$ ightarrow\$ Admin reviews in `AdminProperties.jsx` \$ ightarrow\$ Upon approval, status changes to `'available'` and listing appears on public `properties.jsx` search.
- 2

Seller Role Elevation Workflow

Buyer submits request to sell \$ ightarrow\$ Document created in `joinRequest.model.js` \$ ightarrow\$ Buyer sees `PendingApproval.jsx` \$ ightarrow\$ Admin approves in `SellerRequests.jsx` \$ ightarrow\$ User's `role` updated to `'seller'` in MongoDB \$ ightarrow\$ Unlocks `SellerDashboard.jsx` .
- 3

Real-Time WebSocket Inquiry & Chat Workflow

Buyer clicks "Send Inquiry" on `PropertyDetails.jsx` \$ ightarrow\$ Backend creates `Inquiry` & `Conversation` \$ ightarrow\$ Socket.io emits `new_message` event to seller \$ ightarrow\$ Seller opens `ChatMessages.jsx` \$ ightarrow\$ Live 1-on-1 communication with typing indicators.

DOCUMENT SUMMARY

This technical document covers the complete non-AI codebase of the EstateEase platform. All models, controllers, client panels (Buyer, Seller, Admin), public pages, and socket events are fully functional and integrated.